

Design and Analysis of Algorithm

Module II

Let me know, if any typos and other mistakes in the shared ppt slides

Module 2

Module No. 2	Brute Force And Divide-And-Conquer	8 Hours
Brute Force- Traveling Salesman Problem, 0/1-Knapsack Problem - Assignment problem.		
Divide and Conquer methodology, Binary Search, Merge sort, Quick sort.		

- A divide and conquer algorithm repeatedly reduces an instance of a problem to one or more smaller instances of the same problem (usually recursively), until the instances are small enough to solve easily. One such example of divide and conquer is merge sorting.
- Sorting can be done on each segment of data after dividing data into segments and sorting of entire data can be obtained in conquer phase by merging them. A simpler variant of divide and conquer is called decrease and conquer algorithm, that solves an identical sub problem and uses the solution of this sub problem to solve the bigger problem.
- Divide and conquer divides the problem into multiple sub problems and so conquer stage will be more complex than decrease and conquer algorithms. An example of decrease and conquer algorithm is binary search algorithm.

Binary Search Algorithm

Binary search is a fast search algorithm with **run-time complexity of $O(\log n)$** .

This search algorithm works on the **principle of divide and conquer**.

For this algorithm to work properly, **the data collection should be in the sorted form**.

Features of Binary Search

- It is great to search through large sorted arrays.

- It has a time complexity of $O(\log n)$ which is a very good time complexity.

- It has a simple implementation.

Binary Search Algorithm

Search 23

0	1	2	3	4	5	6	7	8	9
2	5	8	12	16	23	38	56	72	91

23 > 16
take 2nd half

L=0	1	2	3	M=4	5	6	7	8	H=9
2	5	8	12	16	23	38	56	72	91

23 > 56
take 1st half

0	1	2	3	4	L=5	6	M=7	8	H=9
2	5	8	12	16	23	38	56	72	91

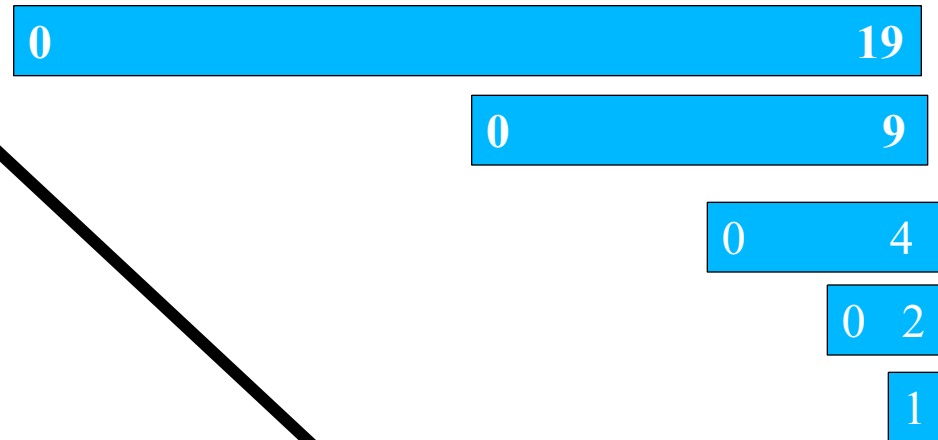
Found 23,
Return 5

0	1	2	3	4	L=5, M=5	H=6	7	8	9
2	5	8	12	16	23	38	56	72	91

Binary Search Algorithm

```

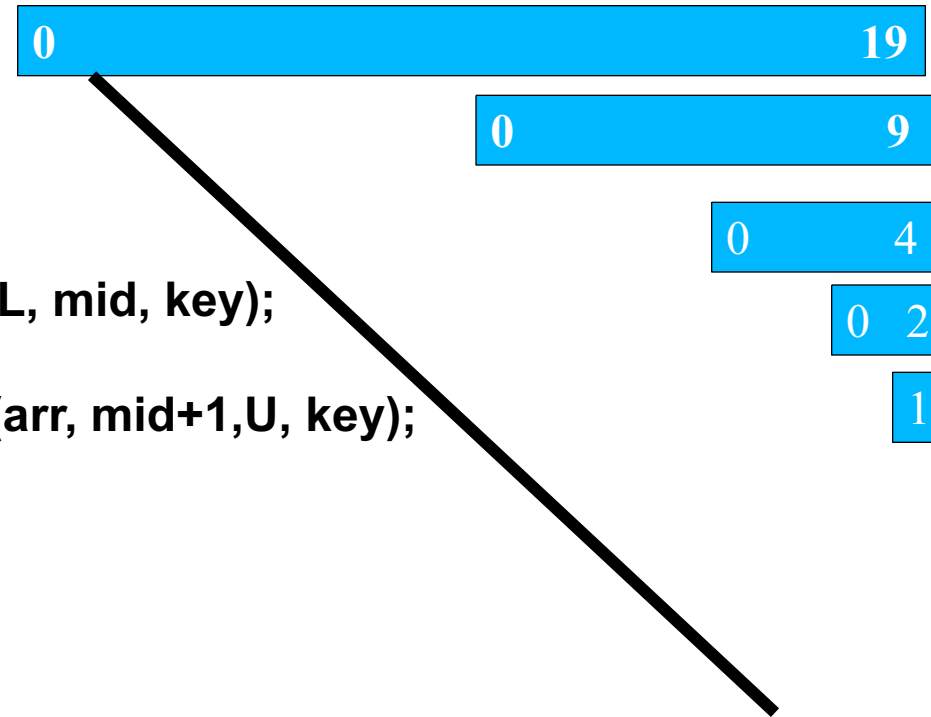
public static int binarySearch(int arr[], int L, int U, int key) {
    int mid ;
    while(L <= U )
    {
        mid = (L + U)/2;
        if ( arr[mid] == key )
            return mid;
        else if ( arr[mid] < key )
            L = mid + 1;
        else
            U = mid - 1;
    }
    if ( L > U )
        return -1;
}
    
```



Binary Search Algorithm

```

public static int BSearch(int arr[], int L, int U, int key)
{
    if(L <= U )
    {
        mid = (L + U)/2;
        if ( arr[mid] == key )
            return mid;
        if( arr[mid] > key)
            return Bsearch(arr, L, mid, key);
        else
            return Bsearch(arr, mid+1,U, key);
    }
    return -1;
}
    
```



1. **Divide Step:** We divide the problem into one or more than one smaller subproblems. Here subproblems are the smaller versions of the same problem for smaller input sizes.
2. **Conquer Step:** We solve each subproblem recursively by calling the same function. Here recursion will automatically handle the work for us.
3. **Combine Step:** We combine the solutions of the subproblems to build the solution of the original problem. For this, we perform some additional operations other than recursive calls. **Note:** In some algorithms, this part is optional.
4. **Base Case:** This represents the smallest version of the problem where recursion will terminate and directly return the solution.

Note: Before learning this strategy, we recommend exploring the concept of recursion.

- $T(n) = aT(n/b) + f(n)$,
where,
 n = size of input
 a = number of subproblems in the recursion
 n/b = size of each subproblem. All
subproblems are assumed to have the
same size.
 $f(n)$ = cost of the work done outside the
recursive call, which includes the cost of
dividing the problem and cost of merging the
solutions

- $T(n) = aT(n/b) + f(n)$,
 $T(n) = T(n/2) + C$
- Use master method to find time complexity
- $a=1$, $b=2$, and $f(n)=C$
- $n^{\log_b a} = C$
- $n^{\log_2 1} = C$
- $n^0 = C$, case 2 is satisfying

$$T(n) = \Theta(n^{\log_b a} \log n)$$

- $T(n) = 1 * \log n \rightarrow (\log n)$

- lets $T(n)$ be total time complexity for worst case n = total number of elements
- $T(n) = T(n/2) + C$
- $T(n) = \mathbf{T(n/4) + C + C}$
- $T(n) = T(n/8) + 3 * C, \quad T(n) = T(n/16) + 4 * C$
- $T(n) = T(n/2^4) + 4 * C$
- $T(n) = T(n/2^k) + k * C$
- Let me take $n = 2^k, \quad T(n) = T(n/n) + k * C$
- $T(n) = T(1) + k$
- $\mathbf{n = 2^k, \text{ and apply } \log_2 \text{ both sides, then it becomes, } \log_2 n = \log_2 2^k}$
- $\mathbf{k = \log_2 n}$
- $\mathbf{T(n) = O(\log_2 n)}$

Merge-sort is based on the **divide-and-conquer approach**.

Divide and Conquer is an **algorithmic pattern**. In algorithmic methods, the design is to take a dispute on a huge input, break the input into minor pieces, decide the problem on each of the small pieces, and then merge the piecewise solutions into a global solution.

It involves the following three steps:

- Divide the array into two (or more) subarrays
- Sort each subarray (Conquer)
- Merge them into one (in a smart way!)

Merge Sort

Consider the array of numbers: **27 10 12 25 34 16 15 31**

Step1: divide it into two parts

27 10 12 25 | **34 16 15 31**

Step2: divide each part into two parts

27 10 | **12 25** | **34 16** | **15 31**

Step3: compare each parts swap it, if not in order

10 | **27** | **12** | **25** | **16** | **34** | **15** | **31**

Step4: merge (cleverly-!) parts

10 27 | **12 25** | **16 34** | **15 31**

Step5: merge parts

10 12 25 27 | **15 16 31 34**

Step6: merge parts into one

10 12 15 16 25 27 31 34

```
MergeSort(arr[ ], L, U)
{
    If( U > L)
    {   middle m = L + (U - L)/2
        mergeSort(arr, L, m)
            mergeSort(arr, m + 1, U)
        merge(arr, L, m, U)
    }
}
```

Merge function

```
void merge(int arr[], int l, int m, int u)
{
    int n1 = m - l + 1;  int n2 = u - m;
    //-----Create temp arrays
    int L[] = new int[n1]; int R[] = new
int[n2];

    //----Copy data to temp arrays
    for (int i = 0; i < n1; ++i)
        L[i] = arr[l + i];
    for (int j = 0; j < n2; ++j)
        R[j] = arr[m + 1 + j];
    //-----Merge the temp arrays
    int i = 0, j = 0;
```

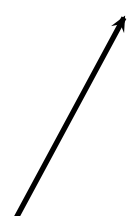
```
// Initial index of merged subarray array
int k = 1;
while (i < n1 && j < n2)
{   if (L[i] <= R[j])
        arr[k] = L[i++];
    else
        arr[k] = R[j++];

    k++;
}
while(i<n1){arr[k++] = L[i++];}
while(j<n2){arr[k++] = R[j++];}
}
```


Recurrence for merge sort



$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1; \\ 2T(n/2) + \Theta(n) & \text{if } n > 1. \end{cases}$$

$$T(n) = \Theta(1) + 2T(n/2) + \Theta(n)$$


MERGE-SORT $A[1 \dots n]$

1. If $n = 1$, done.
2. Recursively sort $A[1 \dots \lceil n/2 \rceil]$ and $A[\lceil n/2 \rceil + 1 \dots n]$.
3. “Merge” the 2 sorted lists

Sloppiness: Should be $T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor)$, but it turns out not to matter asymptotically.

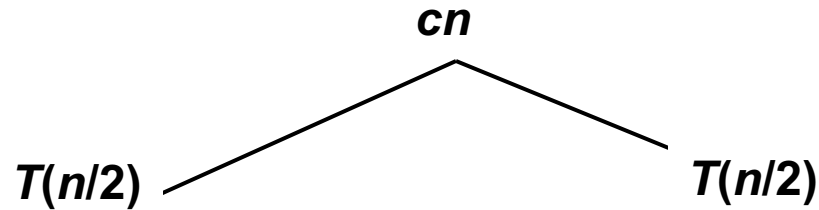
- Lets $T(n)$ be the time complexity for best cases n = total number of elements then
- $T(n) = 2 * T(n/2) + n$
- $2 * T(n/2)$ is because we are dividing array into two array of equal size $constant * n$ is because we will be traversing elements of array in each level of
- **$T(n) = 2 * T(n/2) + n$** further we will divide an array in to two-array of equal size.
- so $T(n) = 2 * [2 * T(n/4) + n/2] + n$
- **$\rightarrow 4 * T(n/4) + 2n$**
- $T(n) = 2^k * T(n/(2^k)) + k * n$
- **$T(n) = n * T(1) + n * \log n = O(n * \log_2(n))$**

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.

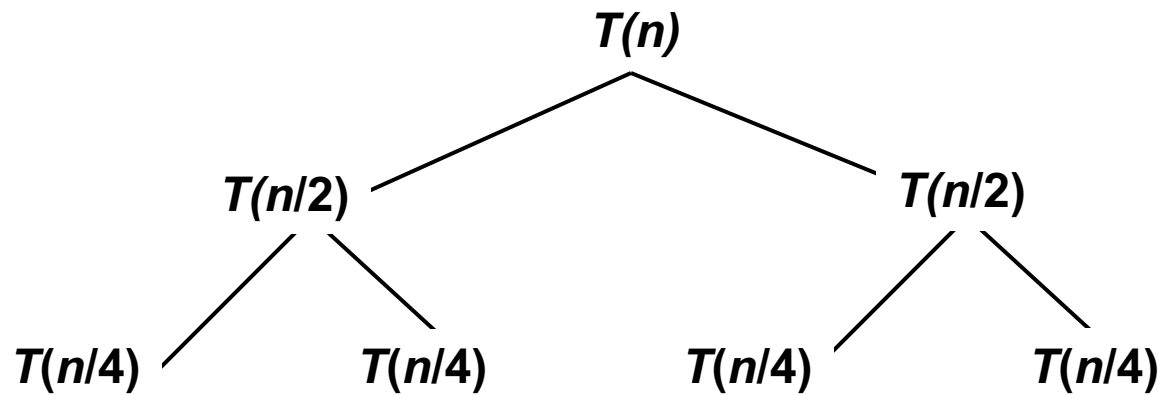
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.

$T(n)$

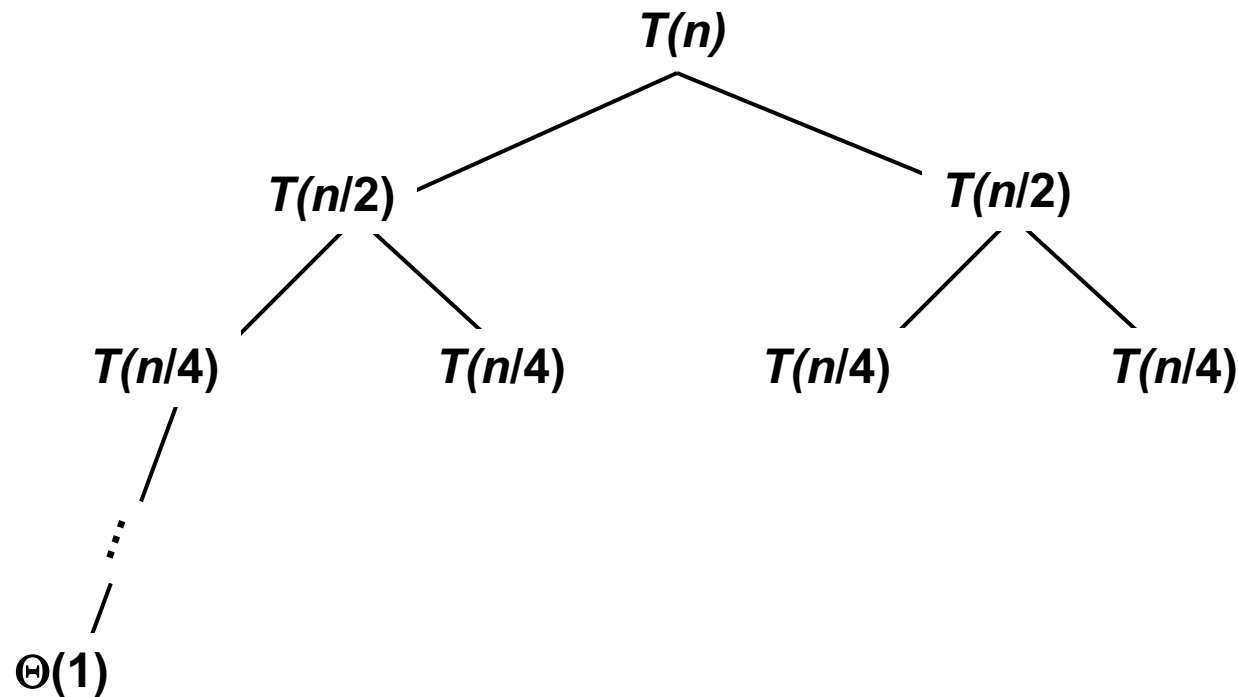
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



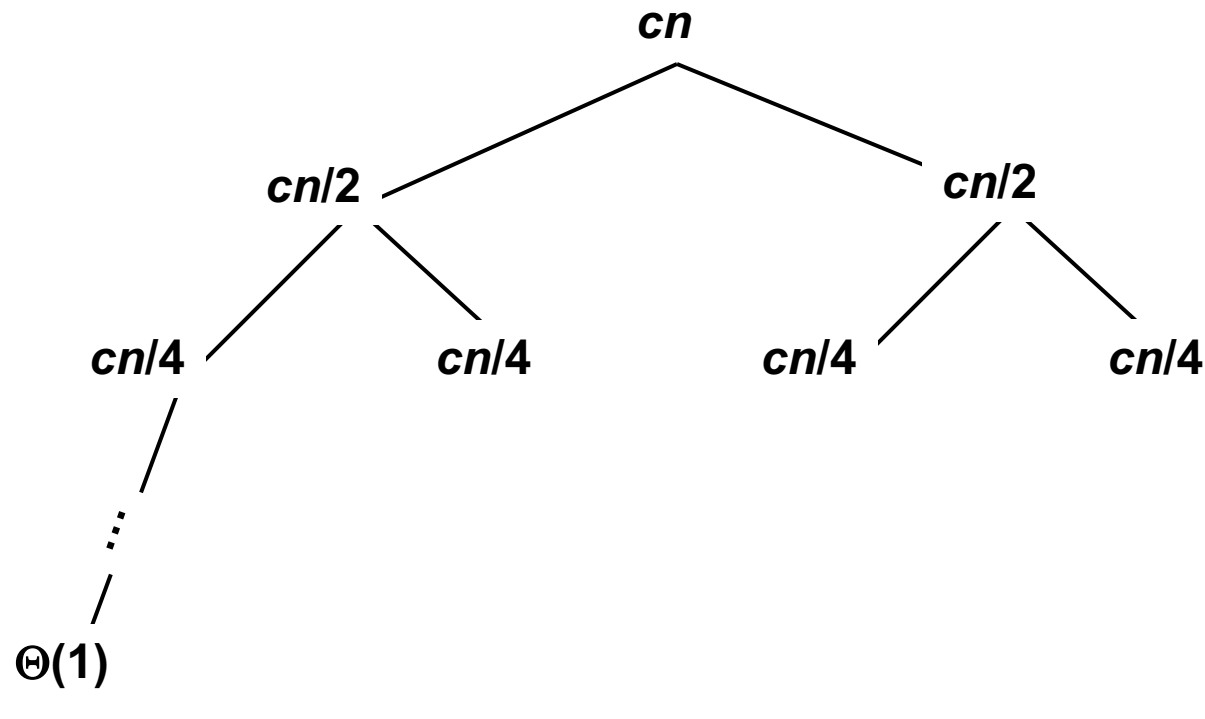
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



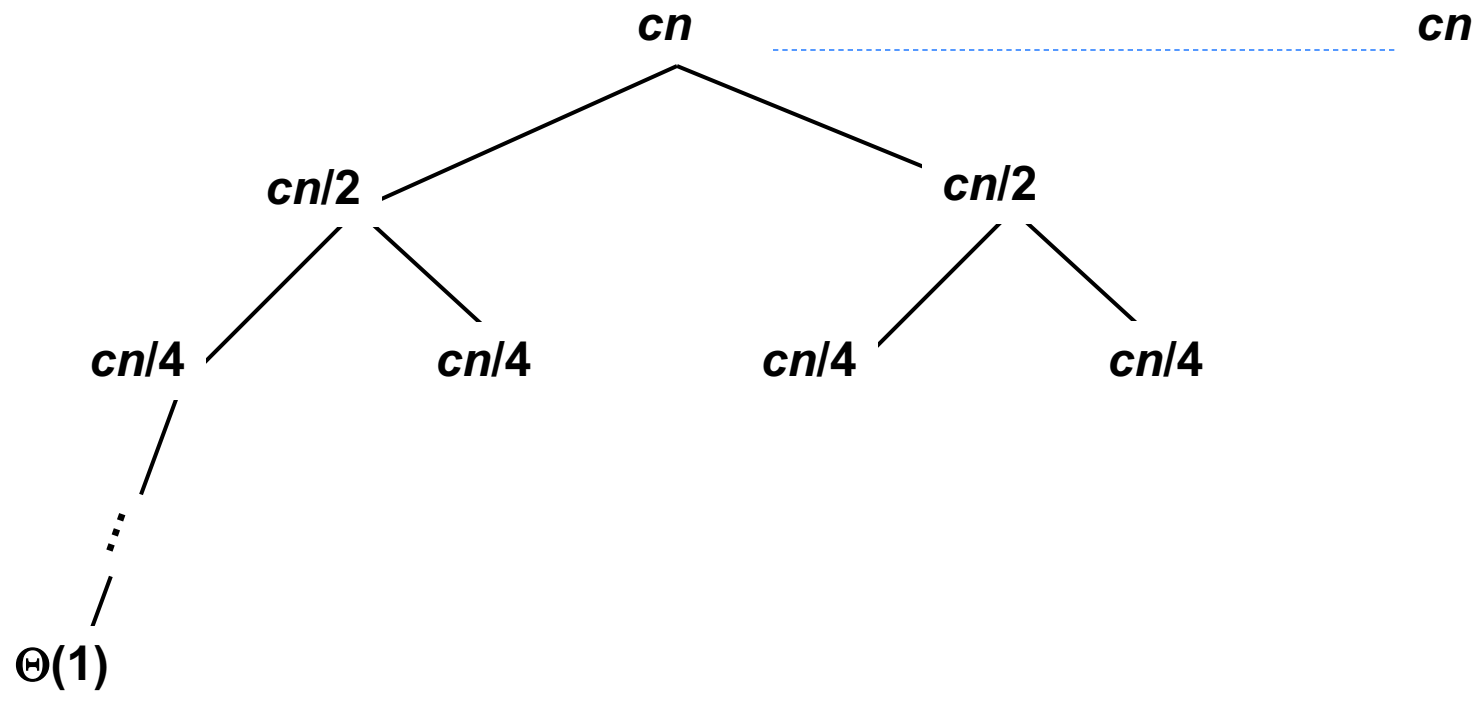
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



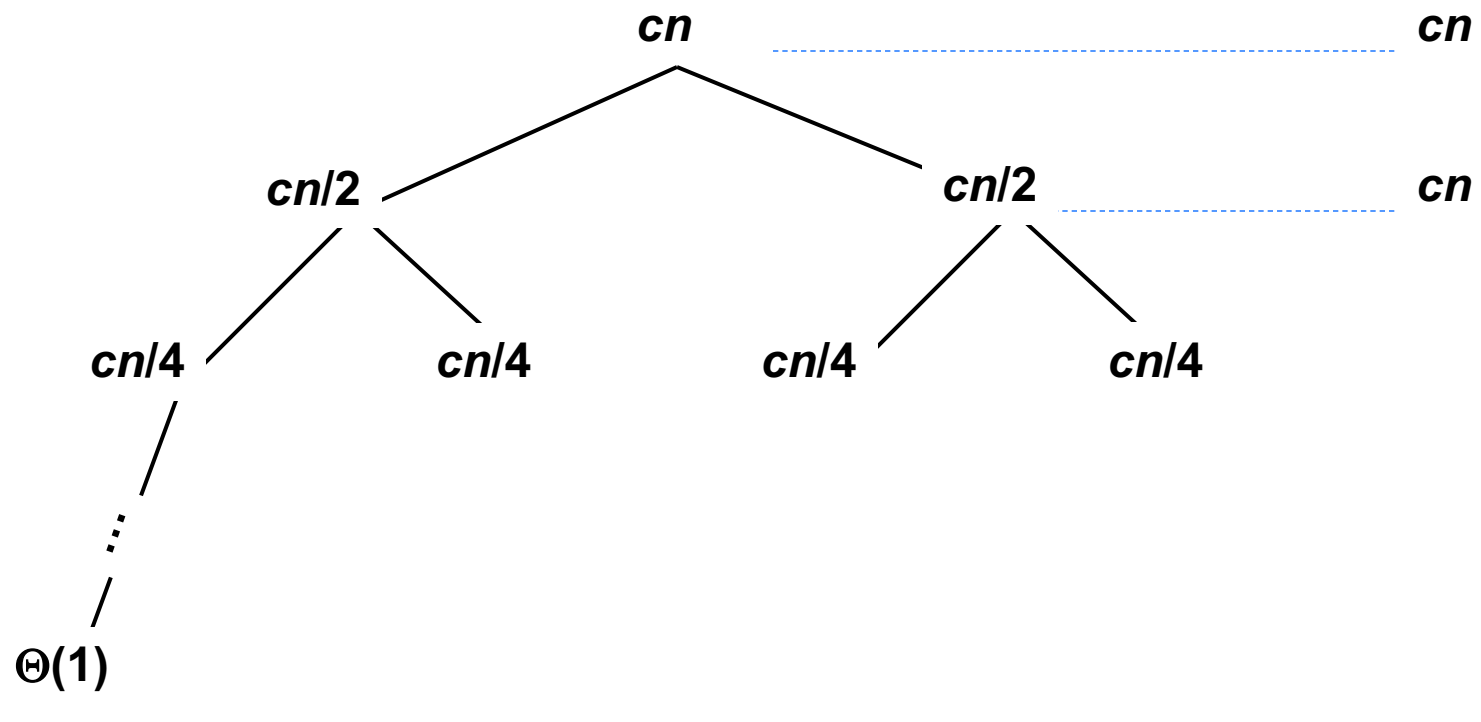
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.

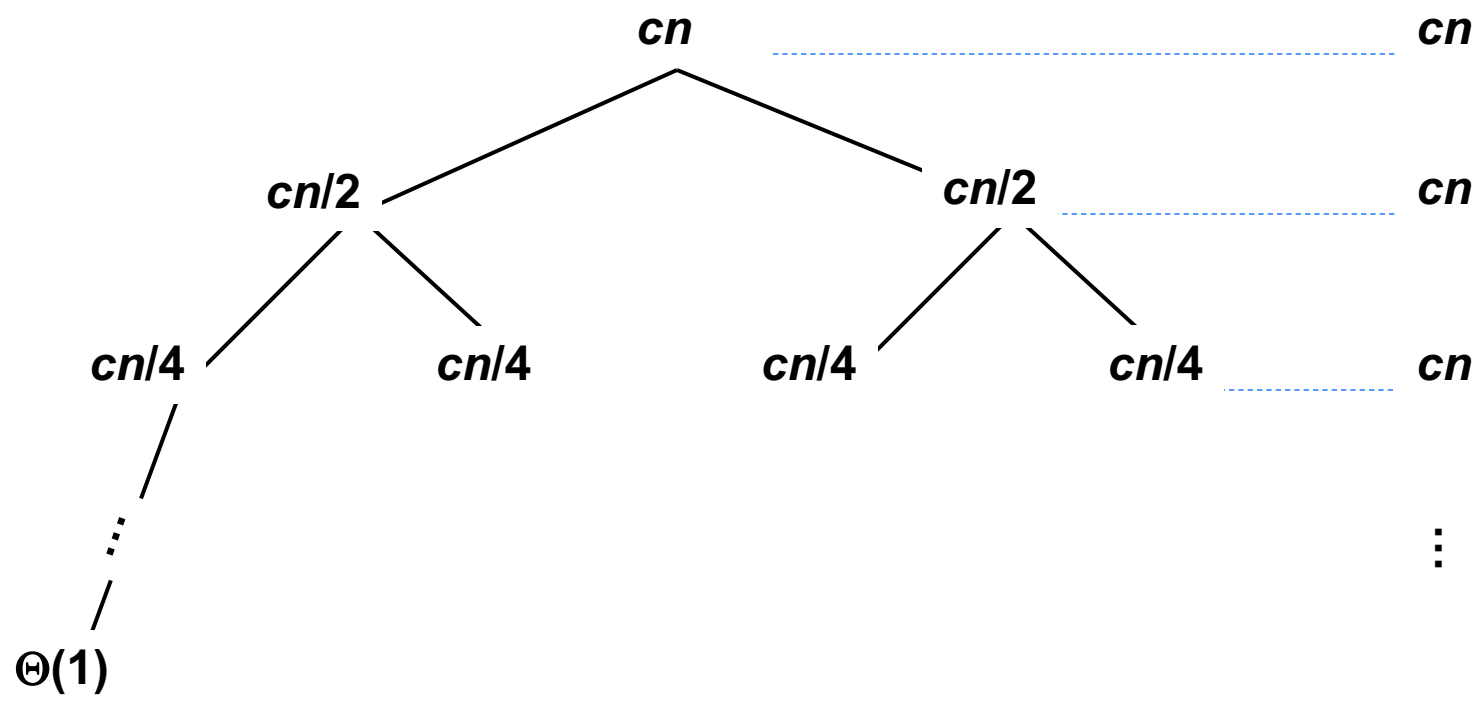


Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



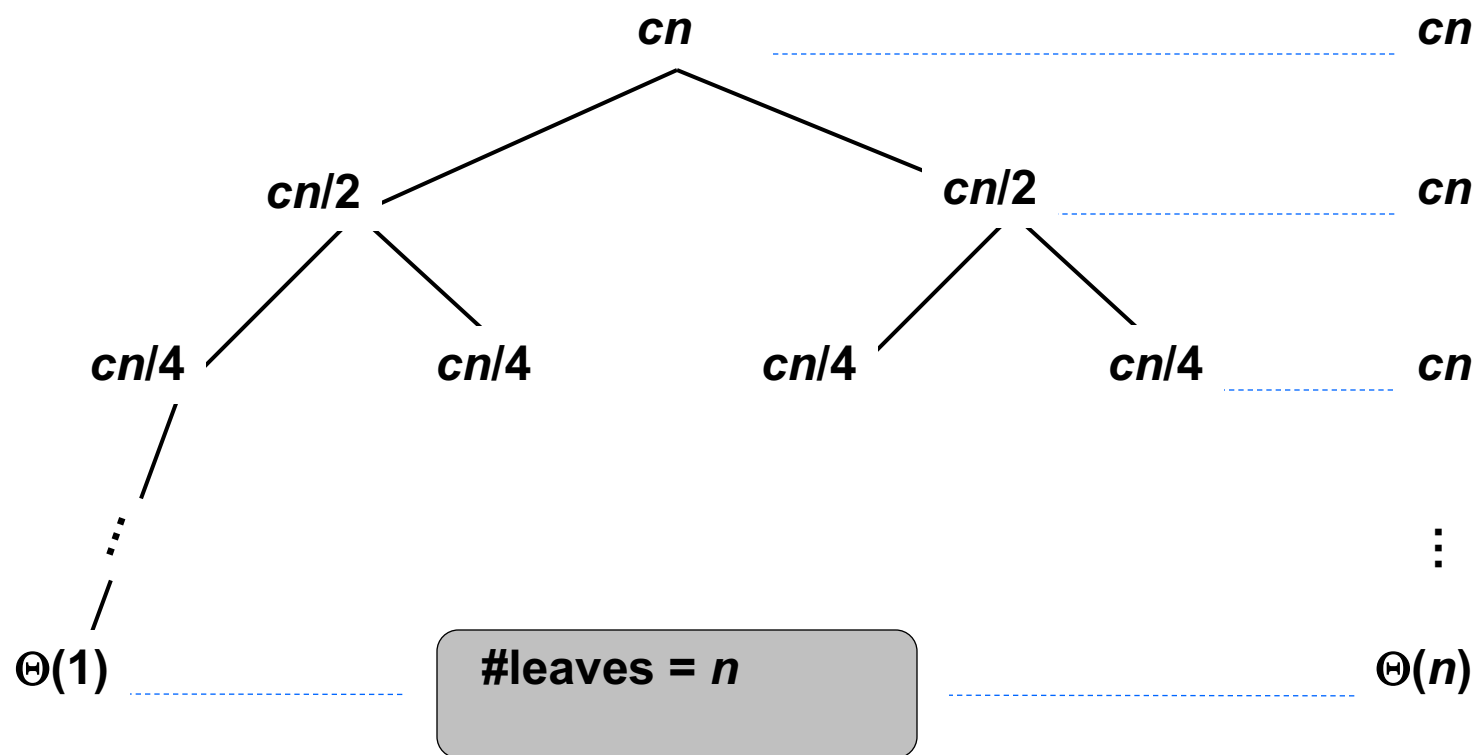
Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



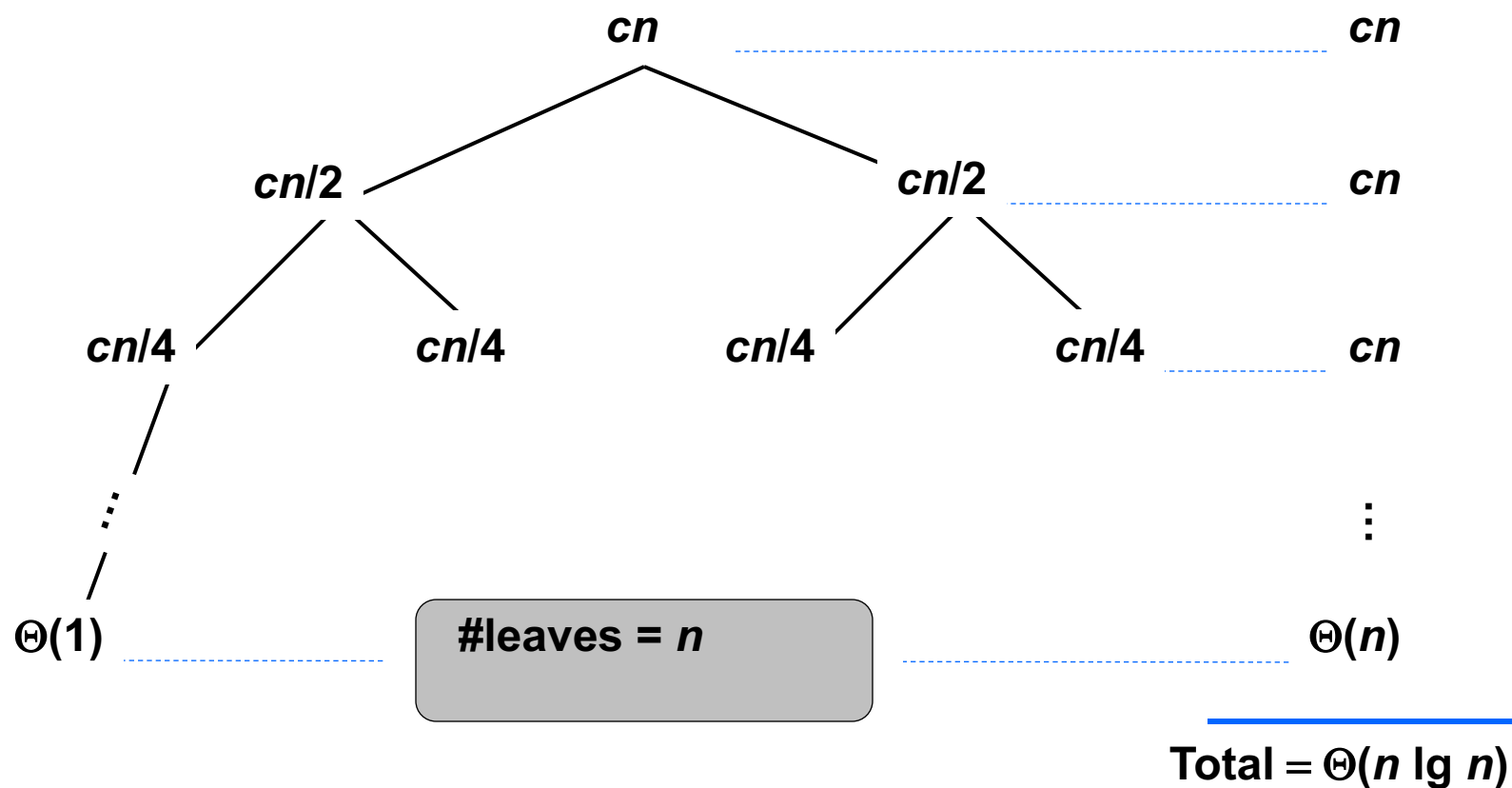
Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



Quick sort is a **fast sorting algorithm** used to sort a list of elements. Quick sort algorithm is invented by **C. A. R. Hoare**.

The quick sort algorithm attempts to separate the list of elements into two parts and then sort each part recursively.

The list is divided into two partitions such that "**all elements to the left of pivot are smaller than the pivot and all elements to the right of pivot are greater than or equal to the pivot**".

In Quick sort algorithm, partitioning of the list is performed using following steps...

Step 1 - Consider the first element of the list as pivot.

Step 2 - Define two variables i and j . Set i and j to first and last elements of the list respectively.

Step 3 - Increment i until $\text{list}[i] > \text{pivot}$ then stop.

Step 4 - Decrement j until $\text{list}[j] < \text{pivot}$ then stop.

Step 5 - If $i < j$ then exchange $\text{list}[i]$ and $\text{list}[j]$.

Step 6 - Repeat steps 3,4 & 5 until $i > j$.

Step 7 - Exchange the pivot element with $\text{list}[j]$ element.

Quick Sort - Step by Step Process



Consider the List: 5 3 8 1 4 6 2 7

P	i					j	
5	3	8	1	4	6	2	7

				P			
4	3	2	1	5	6	8	7

P			i & j
4	3	2	1

P	i					j	
5	3	2	1	4	6	8	7

P	i		j
4	3	2	1

P	i	j
6	8	7

			P
1	3	2	4

P		i				j	
5	3	2	1	4	6	8	7

P		i	j
4	3	2	1

P			
6	7	8	

P	i	j
1	3	2

P			i & j				
5	3	2	1	4	6	8	7

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

P			
1	2	3	

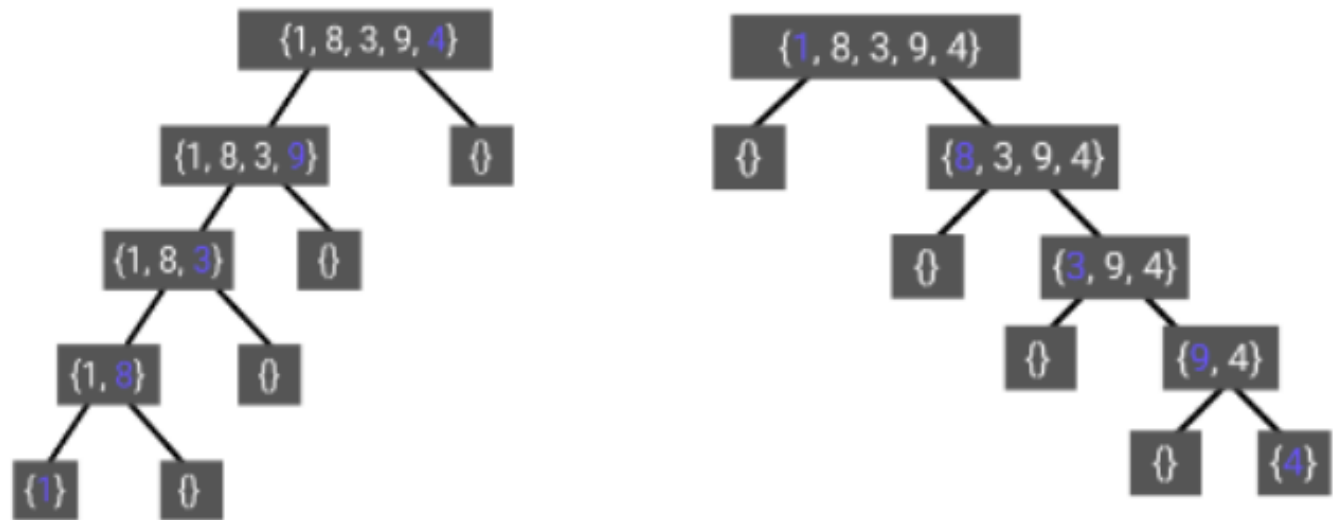
```
QuickSort(arr[ ], low, high) {  
    if (low < high) {  
        pivot = partition(arr, low, high);  
        quickSort(arr, low, pivot - 1);  
        quickSort(arr, pivot + 1, high);  
    }  
}
```

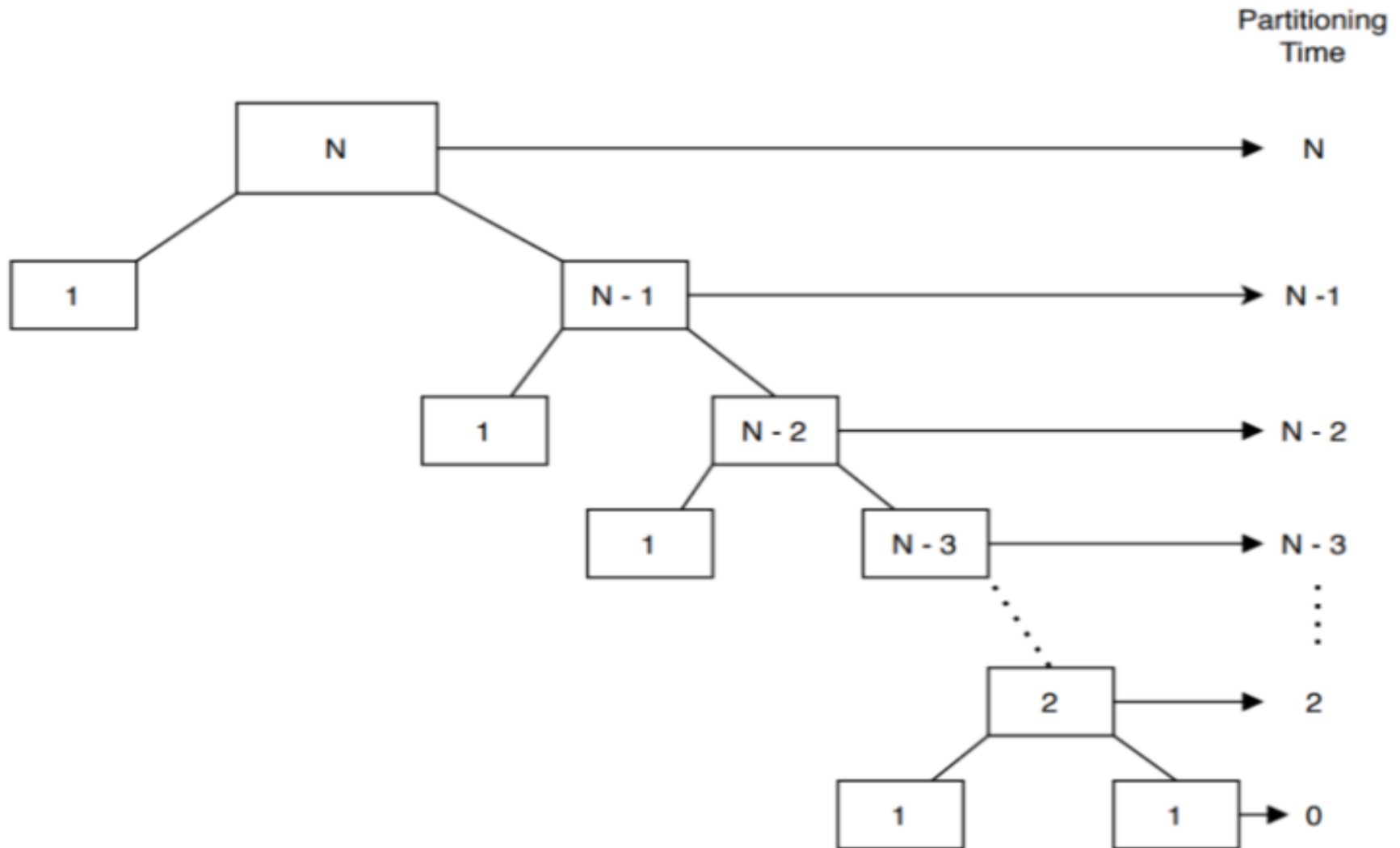
```
Int Partition(int arr[], int low, int high) {  
    int pivot = arr[low], int i = low+1, j = high;  
    while(i<=j)  
    {    while(arr[i] < pivot && i<high) { i ++; }  
        while (arr[j] > pivot && j>low) { j--; }  
        if (i < j)  
            swap(arr[i], arr[j]);  
    }  
    swap(arr[j], arr[low]);  
    return j;  
    }
```

- Lets $T(n)$ be the time complexity for best cases n = total number of elements then
- $T(n) = 2 * T(n/2) + n$
- $2 * T(n/2)$ is because we are dividing array into two array of equal size $constant * n$ is because we will be traversing elements of array in each level of
- **$T(n) = 2 * T(n/2) + n$** further we will divide an array in to two-array of equal size.
- so $T(n) = 2 * [2 * T(n/4) + n/2] + n$
-

- $\rightarrow 4 * T(n/4) + 2n$
- $\rightarrow 8T(n/8) + 3n$
- $2^3 T(n/2^3) + 3n$
-after k steps
- $T(n) = 2^k * T(n/(2^k)) + k * n$
- Let $n = 2^k$
- $T(n) = nT(n/n) + k * n \rightarrow nT(1) + k * n$
- find value of k by applying log both sides of $n = 2^k$
- $k = \log n$
- **$T(n) = n + n * \log n = O(n * \log_2(n))$**

Worst Case Time Complexity of Quick Sort





- $T(n) = T(J) + T(n-J) + M(n)$
- Time Complexity for N elements = Time Complexity for J elements + Time Complexity for n-J elements + Time Complexity for finding the pivot

Quick Sort: Worst case

- lets $T(n)$ be total time complexity for worst case n = total number of elements $T(n) = T(1) + T(n-1) + n$
- $T(n) = [T(n-2) + (n-1)] + n$
- $T(n) = T(n-2) + (n-1) + n$
- $T(n) = T(n-3) + (n-2) + (n-1) + n$
- $T(n) = T(n-k) + (n - (k-1)) + \dots + (n-3) + (n-2) + (n-1) + n$
- Put $k=n$
- $T(n) = T(n-n) + 1 + 2 + 3 + 4 + \dots + (n-1) + n$
- $T(n) = T(0) + [n*(n+1)]/2$
- removing constant terms $T(n) = n*n + n/2$
- **$T(n) = O(n^2)$**

Example 1

- $T(n) = 4T(n/2) + \text{sqrt}(n)$
- $a=4$, $b=2$ and $f(n) = n^{1/2}$
- $f(n) = n^{\log_b a}$
- $n^{1/2} = n^{\log_2 4}$
- $n^{1/2} = n^{2-E}$ case 1 is satisfying
- $T(n) = O(n^2)$

Example 2: $4T(n/2) + n^2 \sqrt{n}$

- $T(n) = 4T(n/2) + n^2 \sqrt{n}$
- $a=4$, $b=2$ and $f(n) = n^{5/2}$
- $f(n) = n^{\log_b a}$
- $n^{5/2} = n^{\log_2 4}$
- $n^{5/2} = n^{2+E}$ case 3 is satisfying
- $T(n) = O(n^{5/2})$

Example 3



- $T(n) = T(n - 1) + \log n$
 - $T(n-1) = T(n-2) + \log(n-1)$
 - $T(n-2) = T(n-3) + \log(n-2)$
- $T(n) = T(n-3) + \log(n-2) + \log(n-1) + \log n$

$$T(n) = T(1) + \sum_{k=2}^n \log k$$

$$\sum_{k=2}^n \log k = \log 2 + \log 3 + \cdots + \log n = \log(2 \cdot 3 \cdot \cdots \cdot n) = \log(n!)$$

Thus:

$$T(n) = T(1) + \log(n!)$$

Example 4

$$T(n) = 3T(n/2) + n \log n$$

Where $a=3, b=2$ $f(n) = n \log n$

$$n^{\log_b a} = n^{\log_2 3} \rightarrow n^{1.585}$$

The function $f(n) = n \log n$ grows slower than $n^{1.585}$, because the logarithmic term only adds a lower-order growth.

$$T(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$$

Example 5

$$T(n) = nT(n-1) \text{ and } T(1) = 1$$

$$= n[(n-1)T(n-2)]$$

$$= n(n-1)[(n-2)T(n-3)]$$

$$= n(n-1)(n-2)[(n-3)T(n-4)]$$

.....after k-1 steps

$$= n(n-1)(n-2)(n-3)(n-4) \dots [(n-(k-1))T(n-k)]$$

Let $n=k-1$

$$T(n) = n(n-1)(n-2)(n-3)(n-4) \dots [(n-(n-2)) * T(n-n)]$$

$$T(n) = n * (n-1) * (n-2) * (n-3) * (n-4) * \dots * (2) * T(1)$$

$$T(n) = n * (n-1) * (n-2) * (n-3) * (n-4) * \dots * (2) * (1)$$

Example 6



- $T(n)=4T(n/2)+n^2$

Example 7



- $T(n) = T(2n/3) + 1$
- $a = 1, b = 3/2, f(n) = 1$
- $F(n) = n^{\log_b a}$
- $n^0 = n^{\log_{3/2} 1}$
- $n^0 = n^0$

case 2 is satisfying

$$T(n) = n^{\log_b a} * \log n$$

$$T(n) = \log n$$

To analyze $T(n)$, compute $p = \log_b a$, and compare $f(n)$ with n^p :

1. **Case 1 (Dominated by recursion):**

If $f(n) = O(n^{p-\epsilon})$ for some constant $\epsilon > 0$, then:

$$T(n) = \Theta(n^p).$$

2. **Case 2 (Balanced growth):**

If $f(n) = \Theta(n^p \cdot \log^k n)$ for some constant $k \geq 0$, then:

$$T(n) = \Theta(n^p \cdot \log^{k+1} n).$$

3. Case 3 (Dominated by additional work):

If $f(n) = \Omega(n^{p+\epsilon})$ for some constant $\epsilon > 0$, and if the regularity condition holds:

$$a \cdot f\left(\frac{n}{b}\right) \leq c \cdot f(n) \quad \text{for some constant } c < 1 \text{ and sufficiently large } n,$$

then:

$$T(n) = \Theta(f(n)).$$

Example

- $T(n) = 2T(n/2) + n^2$

$$a = 2, b = 2, \text{ so } p = \log_2 2 = 1.$$

$$f(n) = n^2, \text{ and } n^2 = \Omega(n^{p+\epsilon}) \text{ with } \epsilon = 1.$$

$$\text{The regularity condition holds because } 2f(n/2) = \frac{n^2}{2} \leq c \cdot n^2.$$

Example 2



- $T(n) = 2T(n/2) + n \log n$

Example 3



- $T(n) = 2T(n/2) + n/(\log n)$

Example 4



- $T(n) = 5T(n/2) + n^2 \cdot \log n$















A brute force algorithm is a simple, comprehensive search approach that systematically explores every possible option.

It's a generic approach to problem-solving that's employed when the issue is small enough to make an in-depth investigation possible.

However, because of their high complexity, brute force techniques are inefficient for large-scale issues.

Brute Force Approach

- **Methodical:** Brute force algorithms investigate every potential solution to a problem. This involves attempting each option in a specified order.
- **Relevance:** When the problem space is small and easily explorable in a fair length of time, brute force is the most appropriate method.
- The temporal complexity of the algorithm becomes unfeasible for larger issue situations.
- **Not optimization or heuristics:** Brute force algorithms don't use optimization or heuristic approaches. They depend on testing every potential outcome without ruling out any using clever pruning or heuristics.

Pros:

- It is guaranteed to find the correct solution by listing all the possible candidate solutions for the problem.
- It is a generic method and **not limited to any specific domain of problems.**
- The brute force method is **ideal for solving small and simpler problems.**
- It always provides best solution.

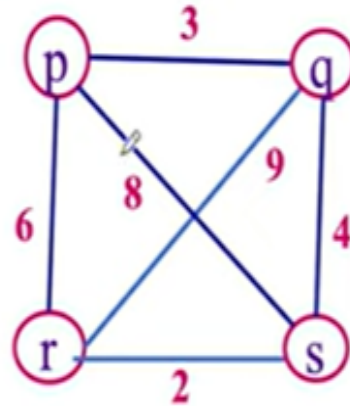
Cons:

- The brute force approach is inefficient.
- For real-time problems, algorithm analysis often goes above the $O(N!)$ order of growth.
- Brute force algorithms are not constructive or creative compared to algorithms that are constructed using some other design paradigms.

The Traveling Salesman Problem (TSP) is the challenge of finding the shortest route for a salesperson to take, given a starting point, a number of cities (nodes), and optionally an ending point.

To solve the TSP using the Brute-Force approach, you must calculate the total number of routes and then draw and list all the possible routes. Calculate the distance of each route and then choose the shortest one

➤ Solve the following TSP which is represented as a graph



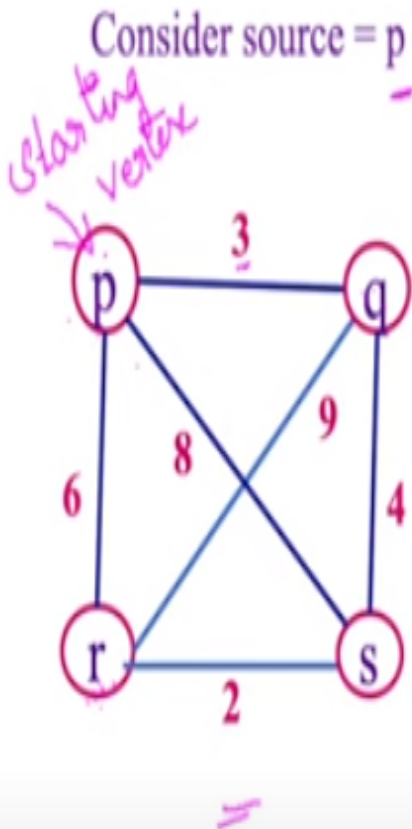
where the cities are represented as p,q,r and s and the distance between various cities are represented as numbers in each of the edge.

Starting node as 'P'

TSP Using Brute Force Approach



1. Find the adjacency Matrix from the given Graph
2. Find all possible path that covers all given points



Adjacency Matrix

	p	q	r	s
p	0	3	6	8
q	3	0	9	4
r	6	9	0	2
s	8	4	2	0

Cost

$p \rightarrow q \rightarrow r \rightarrow s \rightarrow p \Rightarrow 22$

$p \rightarrow q \rightarrow s \rightarrow r \rightarrow p \Rightarrow 15$

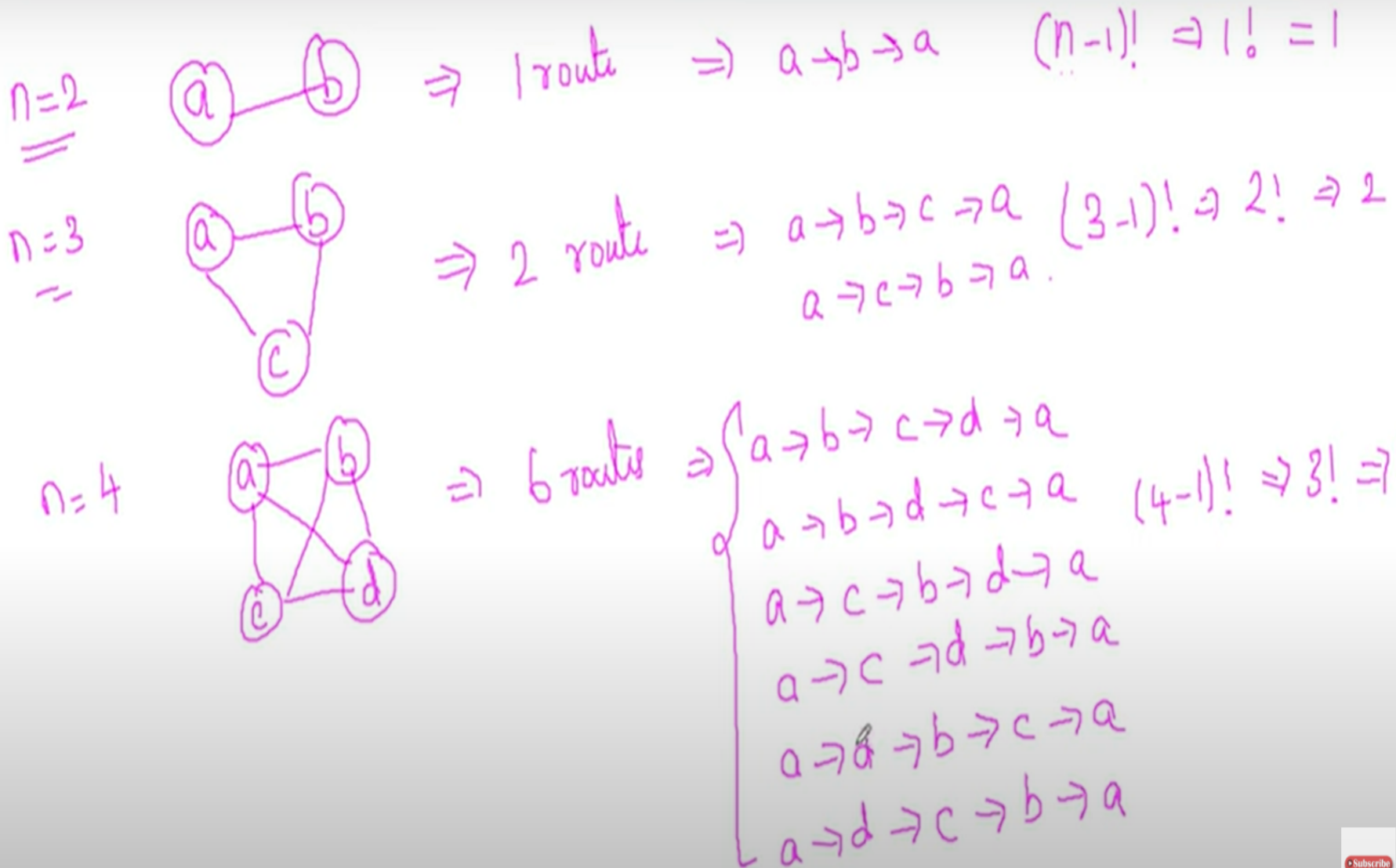
$p \rightarrow r \rightarrow q \rightarrow s \rightarrow p \Rightarrow 27$

$p \rightarrow r \rightarrow s \rightarrow q \rightarrow p \Rightarrow 15$

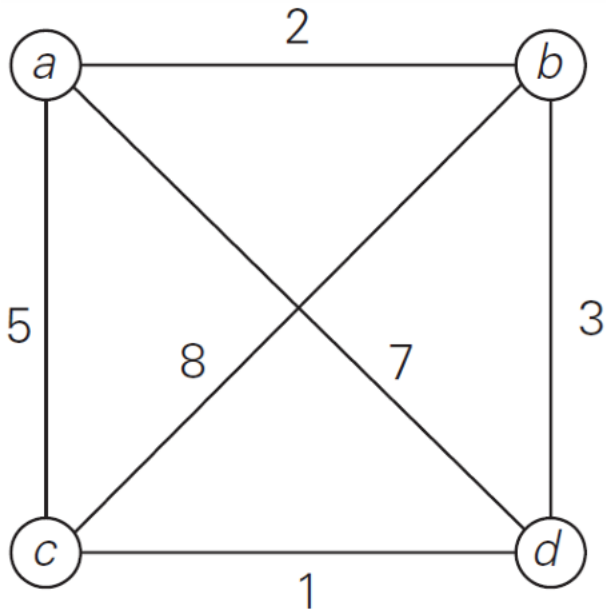
$p \rightarrow s \rightarrow q \rightarrow r \rightarrow p \Rightarrow 27$

$p \rightarrow s \rightarrow r \rightarrow q \rightarrow p \Rightarrow 22$

TSP: Brute Force Approach: Analysis



TSP Using Brute Force

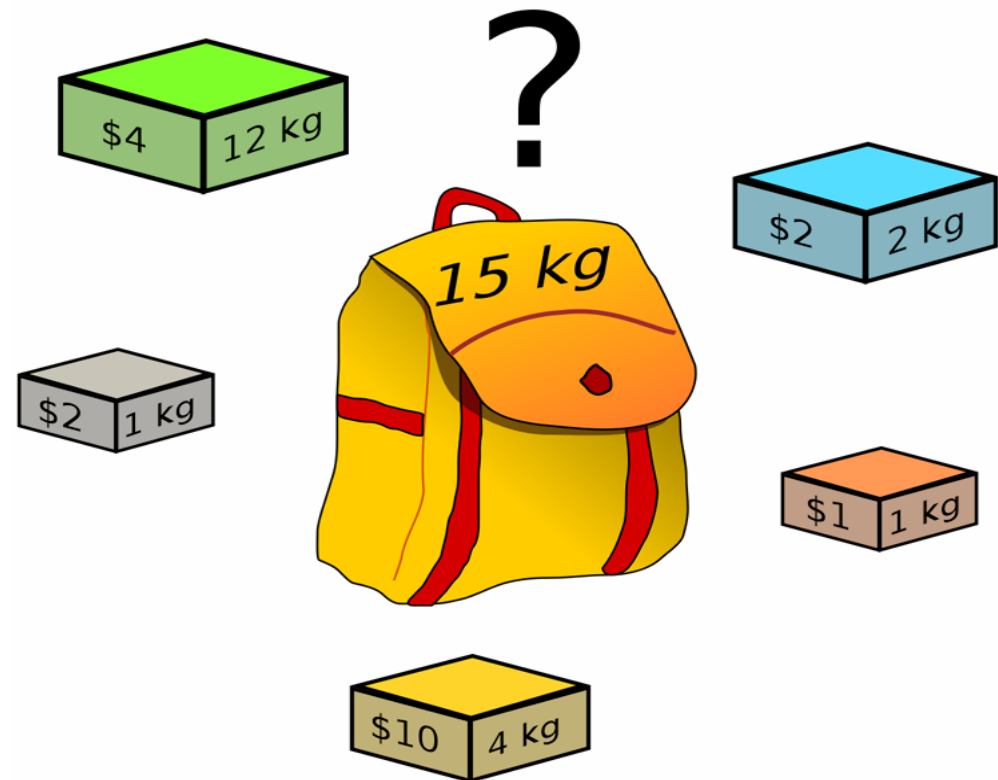


Solution:

<u>Tour</u>	<u>Length</u>	
$a \rightarrow b \rightarrow c \rightarrow d \rightarrow a$	$l = 2 + 8 + 1 + 7 = 18$	
$a \rightarrow b \rightarrow d \rightarrow c \rightarrow a$	$l = 2 + 3 + 1 + 5 = 11$	optimal
$a \rightarrow c \rightarrow b \rightarrow d \rightarrow a$	$l = 5 + 8 + 3 + 7 = 23$	
$a \rightarrow c \rightarrow d \rightarrow b \rightarrow a$	$l = 5 + 1 + 3 + 2 = 11$	optimal
$a \rightarrow d \rightarrow b \rightarrow c \rightarrow a$	$l = 7 + 3 + 8 + 5 = 23$	
$a \rightarrow d \rightarrow c \rightarrow b \rightarrow a$	$l = 7 + 1 + 8 + 2 = 18$	

0/1 Knapsack : Brute Force

Given n items of known weights $w_1, w_2, \dots, w_n$ and values $v_1, v_2, \dots, v_n$ and a knapsack of capacity W . Find the most valuable subset of the items that fit into the knapsack.



0/1 Knapsack Problem



Given a set of n items numbered from 1 up to n , each with a weight w_i and a value v_i , along with a maximum weight capacity W .

Objective is to:

$$\text{maximize } \sum_{i=1}^n v_i x_i$$

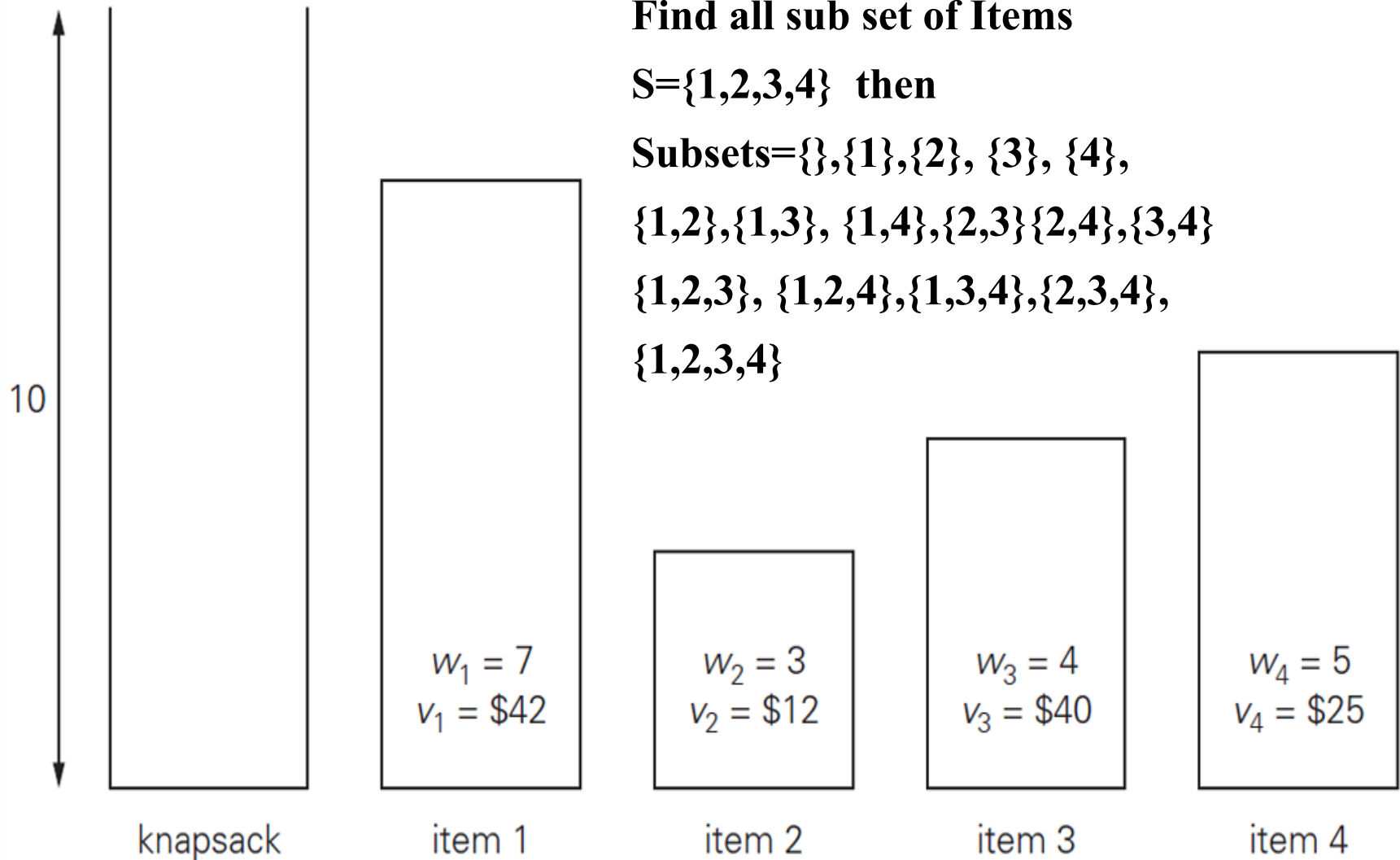
$$\text{subject to } \sum_{i=1}^n w_i x_i \leq W \text{ and } x_i \in \{0, 1\}.$$

where x_i represents the number of instances of item i to include in the knapsack

Brute-Force/Exhaustive Solution

- Generating all the subsets of the set of n items given,
- Compute the total weight of each subset in order to identify feasible subsets (i.e., the ones with the total weight not exceeding the knapsack capacity), and
- **Find a subset with the largest value among them.**

Example



Find all sub set of Items

$S = \{1, 2, 3, 4\}$ then

Subsets = $\{\}, \{1\}, \{2\}, \{3\}, \{4\},$

$\{1, 2\}, \{1, 3\}, \{1, 4\}, \{2, 3\}, \{2, 4\}, \{3, 4\}$

$\{1, 2, 3\}, \{1, 2, 4\}, \{1, 3, 4\}, \{2, 3, 4\},$

$\{1, 2, 3, 4\}$

Subset	Total weight	Total value				
		\$ 0				
		\$42	$w_1 = 7$	$w_2 = 3$	$w_3 = 4$	$w_4 = 5$
		\$12	$v_1 = \$42$	$v_2 = \$12$	$v_3 = \$40$	$v_4 = \$25$
		\$40	item 1	item 2	item 3	item 4
		\$25				
		\$54				
		not feasible				
		not feasible				
		\$52				
		\$37				
		\$65				
		not feasible				
		not feasible				
		not feasible				
		not feasible				
		not feasible				

- Set = $\{1\}$, then subset $\{\}, \{1\} \rightarrow 2$
- Set = $\{1, 2\}$, then subsets are $\{\}, \{1\}, \{2\}, \{1, 2\} \rightarrow 4$
- Set = $\{1, 2, 3\}$, then subsets are
 $\{\}, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\} \rightarrow 8$
- Set = $\{1, 2, 3, 4\}$, then subsets are
 $\{\}, \{1\}, \{2\}, \{3\}, \{4\}, \{1, 2\}, \{1, 3\}, \{1, 4\}, \{2, 3\}, \{2, 4\}, \{3, 4\}, \{1, 2, 3\}, \{1, 2, 4\}, \{1, 3, 4\}, \{2, 3, 4\}, \{1, 2, 3, 4\} \rightarrow 16$
- $T(n) = T(2^n)$

- Since the number of subsets of an n -element set is 2^n , the exhaustive search leads to an algorithm of order **$\Omega(2^n)$** no matter how efficiently individual subsets are generated.

- There are n people (agents) who need to be assigned to execute n tasks, one person per job.
- That is, each person is assigned to exactly one job and each job is assigned to exactly one person.
- The cost that would accrue if the i^{th} person is assigned to the j^{th} job is a known quantity $C[i,j]$ for each pair $i,j = 1, 2, \dots, n$.
- The problem is to find an assignment with the minimum total cost.

A small instance of this problem follows, with the table entries representing the assignment costs $C[i,j]$:

	Job 1	Job 2	Job 3	Job 4
Person 1	9	2	7	8
Person 2	6	4	3	7
Person 3	5	8	1	8
Person 4	7	6	9	4

- The problem is to select one element in each row of the matrix so that all selected elements are in different columns and the total sum of the selected elements is the smallest possible.
- **Assign the jobs to persons in such way that the total cost is minimum.**
- **Note that no obvious strategy for finding a solution works here.**

Brute-Force Approach



	Job 1	Job 2	Job 3	Job 4
Person 1	9	2	7	8
Person 2	6	4	3	7
Person 3	5	8	1	8
Person 4	7	6	9	4

$$C = \begin{bmatrix} 9 & 2 & 7 & 8 \\ 6 & 4 & 3 & 7 \\ 5 & 8 & 1 & 8 \\ 7 & 6 & 9 & 4 \end{bmatrix}$$

Persons

J1 J2 J3 J4

<1, 2, 3, 4>
<1, 2, 4, 3>
<1, 3, 2, 4>
<1, 3, 4, 2>
<1, 4, 2, 3>
<1, 4, 3, 2>

cost = 9 + 4 + 1 + 4 = 18
cost = 9 + 4 + 8 + 9 = 30
cost = 9 + 3 + 8 + 4 = 24
cost = 9 + 3 + 8 + 6 = 26
cost = 9 + 7 + 8 + 9 = 33
cost = 9 + 7 + 1 + 6 = 23

Assignment Problem 2



Problem: In a construction site there are 4 cranes. Each crane must be allocated to one job. The time required for each job for each crane is shown in the table below. Find the best assignment of cranes to the jobs so that the time required to finish the jobs is minimum.

	JOB1	JOB2	JOB3	JOB4
CRANE1	4	2	5	7
CRANE2	8	3	10	8
CRANE3	12	5	4	5
CRANE4	6	3	7	14

- Number Jobs/users =1, possible solutions =1
– J1/u1
- Number Jobs/users =2, possible solutions =2

J1/u1	J2/j2
J1/u2	J2/u1

- Number Jobs/users =3, possible solutions =6

J1/u1	J2/u2	J3/u3
J1/u1	J2/u3	J3/u2
J1/u2	J2/u1	J3/u3
J1/u2	J2/u3	J3/u1
J1/u3	J2/u1	J3/u2
J1/u3	J2/u2	J3/u1

- Number Jobs/users =4, possible solutions =24

- Since the number of permutations to be considered for the general case of the assignment problem is $O(n!)$
- Exhaustive search is impractical for all but very small instances of the problem.